

Ağ Paketlerinin Anatomisi ve Güvenlik Sırları: Wireshark ve TTL'in Rolü

İnternet dünyasında attığımız her adım; bir mesaj göndermek, bir web sitesini ziyaret etmek veya bir sunucuya bağlanmak, aslında milyonlarca küçük "paketin" yola çıkması demektir. Bu paketler, ağ üzerinde hedeflerine ulaşabilmek için **TCP/IP** protokolleri çerçevesinde taşınır.

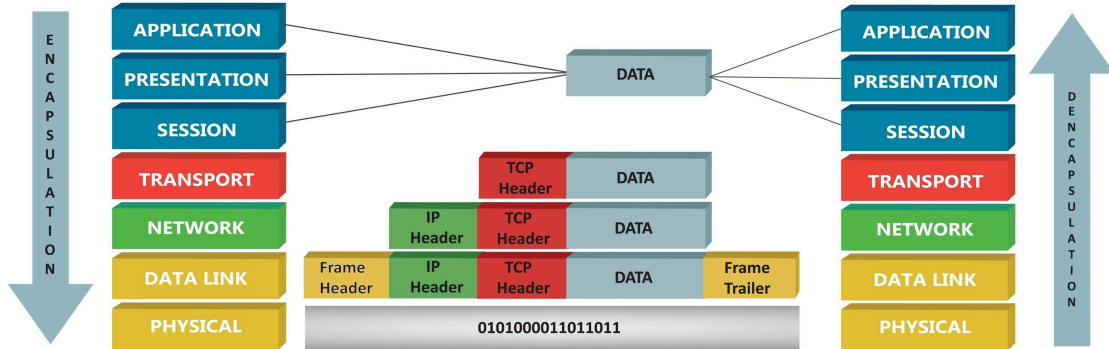
Bu süreci şehir içi bir kurye teslimatına benzetebiliriz:

- **IP Adresi:** Kuryenin gideceği binayı (Hedef Cihaz),
- **Port Numarası:** Teslimatın yapılacağı daireyi (Uygulama/Servis),
- **Header (Başlık):** Paketin taşıma talimatlarını,
- **TTL (Time-to-Live):** Paketin yakıtını veya ömrünü belirler.

Bu yazıda, bir ağ paketinin nasıl oluştuğunu, Wireshark ile bu paketlerin içinin nasıl incelendiğini ve saldırganların bu yapıları nasıl kötüye kullanabildiğini ele alacağız.

1. Kapsülleme (Encapsulation)

Bir veri, bilgisayarımızdan çıkıp internete gidebilmesi için doğrudan gönderilmez; katmanlar hâlinde paketlenerek taşınmaya uygun hale getirilir. Bu sürece **kapsülleme (encapsulation)** adı verilir.



Bu süreci, hassas bir ürünün kargolanması olarak düşünebiliriz:

1. **Data (Uygulama):** Gönderilmek istenen asıl içeriktir (Örn: HTTP isteği).
2. **Segment (Taşıma - TCP/UDP):** Verinin hangi kapıdan (port) gireceği belirlenir.
3. **Packet (Ağ - IP):** Gönderici ve alıcı adresleri (IP) pakete eklenir.
4. **Frame (Veri Bağı - MAC):** Verinin fiziksel ağda hangi cihaza gideceği (MAC adresi) işlenir.

Katman	Veri Birimi	Eklenen Bilgi	Örnek
Uygulama	Data	HTTP isteği	GET /index.html
Taşıma	Segment	Port numarası	80 (HTTP)
Ağ	Packet	IP adresleri	192.168.1.10 → 8.8.8.8
Veri Bağı	Frame	MAC adresleri	00:1A:2B:3C:4D:5E

Bu katmanlı yapı verinin hedefe ulaşmasını sağlarken, saldırganlar için de manipülasyon fırsatları doğurur. Örneğin; IP başlığının değiştirilmesi **IP Spoofing**, MAC adresinin taklit edilmesi ise **MAC Spoofing** saldırılarına zemin hazırlar.

2. Wireshark: Bir Paketin İçeriğini Görmek

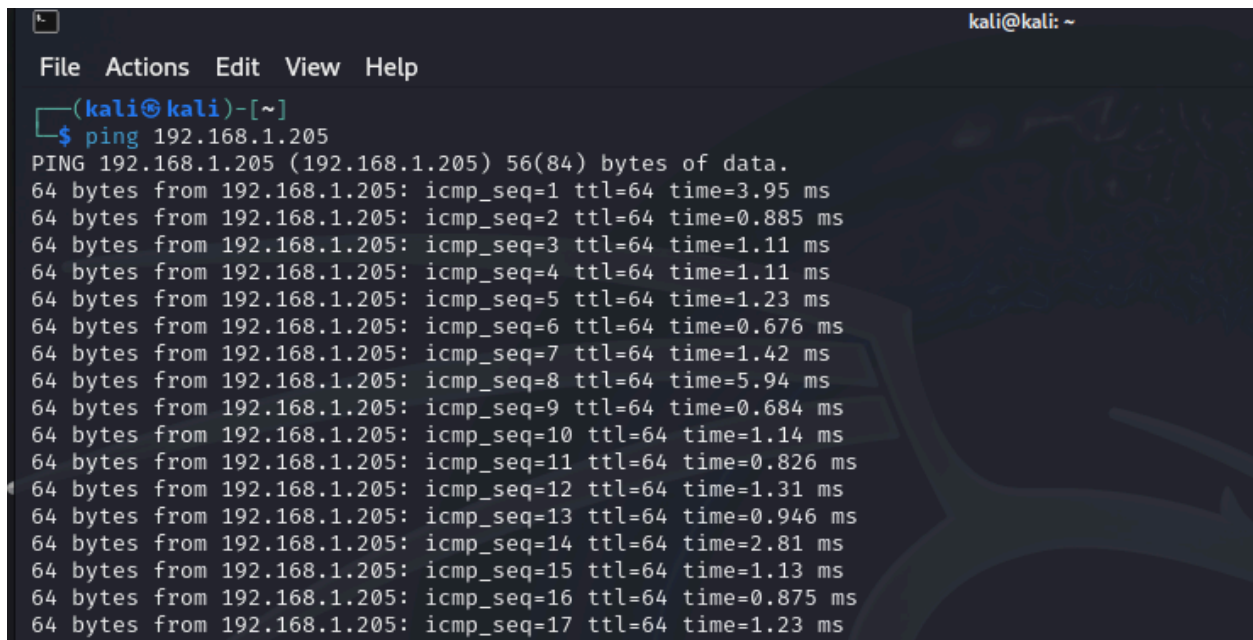
Wireshark, ağ trafiğini yakalayıp paketlerin içindeki başlık bilgilerini detaylı şekilde incelememizi sağlar.

Bir ICMP (ping) paketi incelendiğinde;

- Kaynak ve hedef MAC adresleri,
- Kaynak ve hedef IP adresleri,
- Protokol türü,
- **TTL değeri**

gibi kritik bilgiler görülebilir.

Laboratuvar ortamında Kali Linux makinemden Ubuntu makinesine gönderdiğim ICMP paketlerini Wireshark ile analiz ettiğimde, IP başlığındaki TTL alanı görülüyor

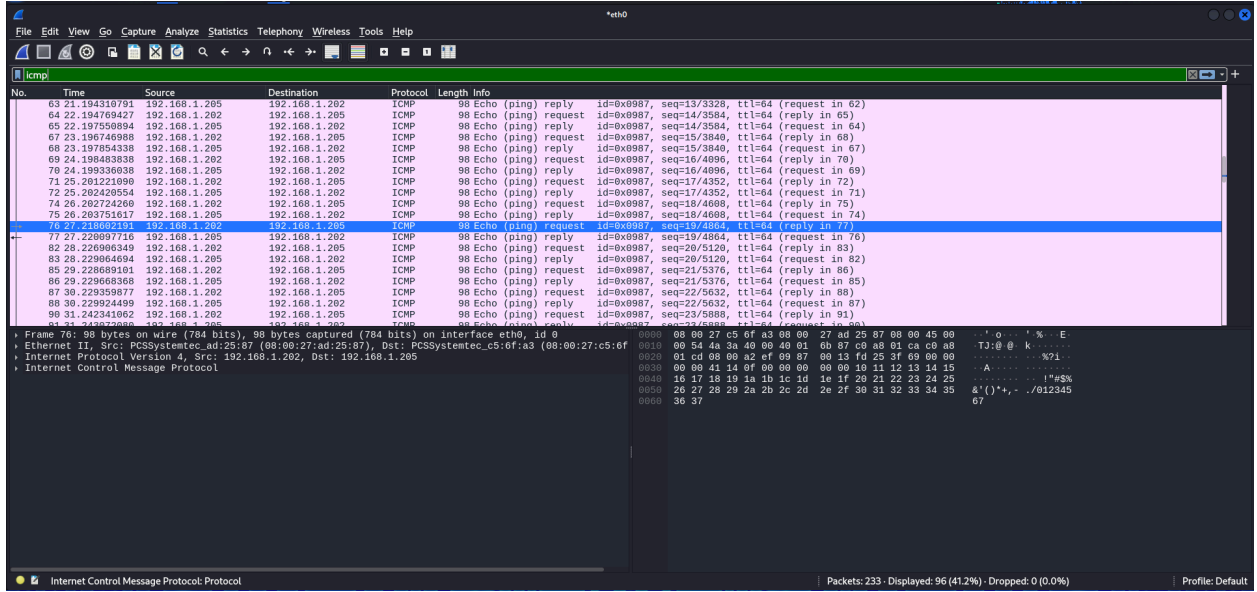


```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ ping 192.168.1.205  
PING 192.168.1.205 (192.168.1.205) 56(84) bytes of data.  
64 bytes from 192.168.1.205: icmp_seq=1 ttl=64 time=3.95 ms  
64 bytes from 192.168.1.205: icmp_seq=2 ttl=64 time=0.885 ms  
64 bytes from 192.168.1.205: icmp_seq=3 ttl=64 time=1.11 ms  
64 bytes from 192.168.1.205: icmp_seq=4 ttl=64 time=1.11 ms  
64 bytes from 192.168.1.205: icmp_seq=5 ttl=64 time=1.23 ms  
64 bytes from 192.168.1.205: icmp_seq=6 ttl=64 time=0.676 ms  
64 bytes from 192.168.1.205: icmp_seq=7 ttl=64 time=1.42 ms  
64 bytes from 192.168.1.205: icmp_seq=8 ttl=64 time=5.94 ms  
64 bytes from 192.168.1.205: icmp_seq=9 ttl=64 time=0.684 ms  
64 bytes from 192.168.1.205: icmp_seq=10 ttl=64 time=1.14 ms  
64 bytes from 192.168.1.205: icmp_seq=11 ttl=64 time=0.826 ms  
64 bytes from 192.168.1.205: icmp_seq=12 ttl=64 time=1.31 ms  
64 bytes from 192.168.1.205: icmp_seq=13 ttl=64 time=0.946 ms  
64 bytes from 192.168.1.205: icmp_seq=14 ttl=64 time=2.81 ms  
64 bytes from 192.168.1.205: icmp_seq=15 ttl=64 time=1.13 ms  
64 bytes from 192.168.1.205: icmp_seq=16 ttl=64 time=0.875 ms  
64 bytes from 192.168.1.205: icmp_seq=17 ttl=64 time=1.23 ms
```

Kali Linux makinesi üzerinden Ubuntu makinesine ICMP isteği gönderdim. Bunun için kullandığım komut: ping 192.168.1.205

Bu komut, hedef sistemin ağ üzerinden erişilebilir olup olmadığını test etmekle birlikte, arka planda ICMP Echo Request ve Echo Reply paketlerinin oluşturulmasını sağlar.

Ping işlemi sırasında oluşan ağ trafiğini, Kali Linux üzerinde **Wireshark** kullanarak yakaladım. Filtreleme işlemi için "icmp" filtresi kullandım ve yalnızca ICMP paketleri inceledim.



No.	Time	Source	Destination	Protocol	Length	Info
63	21.194310791	192.168.1.205	192.168.1.202	ICMP	98	Echo (ping) reply id=0x0987, seq=13/3328, ttl=64 (request in 62)
64	22.194769427	192.168.1.202	192.168.1.205	ICMP	98	Echo (ping) request id=0x0987, seq=14/3584, ttl=64 (request in 64)
65	22.197556994	192.168.1.205	192.168.1.202	ICMP	98	Echo (ping) reply id=0x0987, seq=14/3584, ttl=64 (request in 64)
67	23.196746988	192.168.1.202	192.168.1.205	ICMP	98	Echo (ping) request id=0x0987, seq=15/3840, ttl=64 (request in 68)
68	23.197854338	192.168.1.205	192.168.1.202	ICMP	98	Echo (ping) reply id=0x0987, seq=15/3840, ttl=64 (request in 67)
69	24.198483838	192.168.1.202	192.168.1.205	ICMP	98	Echo (ping) request id=0x0987, seq=16/4096, ttl=64 (request in 70)
70	24.199336038	192.168.1.205	192.168.1.202	ICMP	98	Echo (ping) reply id=0x0987, seq=16/4096, ttl=64 (request in 69)
71	25.201221090	192.168.1.202	192.168.1.205	ICMP	98	Echo (ping) request id=0x0987, seq=17/4352, ttl=64 (request in 72)
72	25.202420554	192.168.1.205	192.168.1.202	ICMP	98	Echo (ping) reply id=0x0987, seq=17/4352, ttl=64 (request in 71)
74	26.202724260	192.168.1.202	192.168.1.205	ICMP	98	Echo (ping) request id=0x0987, seq=18/4608, ttl=64 (request in 75)
75	26.203751617	192.168.1.205	192.168.1.202	ICMP	98	Echo (ping) reply id=0x0987, seq=18/4608, ttl=64 (request in 74)
77	27.220997716	192.168.1.205	192.168.1.202	ICMP	98	Echo (ping) request id=0x0987, seq=19/4864, ttl=64 (request in 76)
82	28.226986349	192.168.1.202	192.168.1.205	ICMP	98	Echo (ping) reply id=0x0987, seq=20/5120, ttl=64 (request in 83)
83	28.229684694	192.168.1.205	192.168.1.202	ICMP	98	Echo (ping) request id=0x0987, seq=20/5120, ttl=64 (request in 82)
85	29.228689181	192.168.1.202	192.168.1.205	ICMP	98	Echo (ping) reply id=0x0987, seq=21/5376, ttl=64 (request in 86)
86	29.229686368	192.168.1.205	192.168.1.202	ICMP	98	Echo (ping) request id=0x0987, seq=21/5376, ttl=64 (request in 85)
87	30.229359877	192.168.1.202	192.168.1.205	ICMP	98	Echo (ping) reply id=0x0987, seq=22/5632, ttl=64 (request in 88)
88	30.229924499	192.168.1.205	192.168.1.202	ICMP	98	Echo (ping) request id=0x0987, seq=22/5632, ttl=64 (request in 87)
90	31.242341062	192.168.1.202	192.168.1.205	ICMP	98	Echo (ping) reply id=0x0987, seq=23/5888, ttl=64 (request in 91)

Frame 70: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth0, id 0
Ethernet II, Src: PCSSystemtec_ad:25:87 (08:00:27:ad:25:87), Dst: PCSSystemtec_c5:6f:a3 (08:00:27:c5:6f:a3)
Internet Protocol Version 4, Src: 192.168.1.202, Dst: 192.168.1.205
Internet Control Message Protocol

Packets: 233 · Displayed: 96 (41.2%) · Dropped: 0 (0.0%) · Profile: Default

Yakalanan paketlerden birini seçip **IP başlığı** detaylarını analiz edebiliriz. Bu başlık içerisinde yer alan **Time-to-Live (TTL)** alanına dikkat edebilirsiniz.

```
Wireshark - Packet 76 - eth0
Frame 76: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth0, id 0
  Ethernet II, Src: PCSSystemtec_ad:25:87 (08:00:27:ad:25:87), Dst: PCSSystemtec_c5:6f:a3 (08:00:27:c5:6f:a3)
    Destination: PCSSystemtec_c5:6f:a3 (08:00:27:c5:6f:a3)
    Source: PCSSystemtec_ad:25:87 (08:00:27:ad:25:87)
    Type: IPv4 (0x0800)
  Internet Protocol Version 4, Src: 192.168.1.202, Dst: 192.168.1.205
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
    Identification: 0x4a3a (19002)
    010. .... = Flags: 0x2, Don't fragment
    0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0x6b87 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.1.202
    Destination Address: 192.168.1.205
  Internet Control Message Protocol

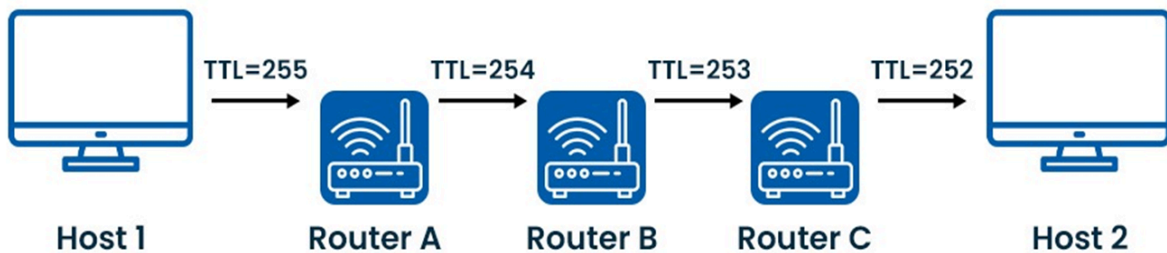
0000  08 00 27 c5 6f a3 08 00 27 ad 25 87 08 00 45 00  ..'.o...%...E
0010  00 54 4a 3a 40 00 40 01 6b 87 c0 a8 01 ca c0 a8  TJ:@ @.k.....
0020  01 cd 08 00 a2 ef 09 87 00 13 fd 25 3f 69 00 00  ..A.....%?i...
0030  00 00 41 14 0f 00 00 00 00 00 10 11 12 13 14 15  ..A.....!#$%
0040  16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25  &'()*+,-./012345
0050  26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35  67
0060  36 37

No.: 76 - Time: 27.218602191 - Source: 192.168.1.202 - Destination: 192.168.1.205 - Protocol: ICMP - Length: 98 - Info: Echo (ping) request id=0x0987, seq=19/4864, ttl=64 (reply in 77)
✓ Show packet bytes
```

Bu analizde, hedef sistemden dönen paketin TTL değerinin **64** olduğu görülüyor. Bu değer, genellikle **Linux/Unix tabanlı işletim sistemleri** tarafından kullanılan varsayılan TTL değeridir.

3. TTL (Time-to-Live): Paketin Ömrü

TTL, bir paketin ağ üzerinde kaç yönlendiriciden geçebileceğini belirleyen bir sayaçtır. Her router geçişinde bu değer 1 azalır ve 0'a ulaştığında paket imha edilir. Bu mekanizma, ağda sonsuz döngülerin oluşmasını engeller.



Ayrıca TTL, sadece bir sayaç değil, aynı zamanda bir **Parmak İzidir**. İşletim sistemleri paket oluştururken farklı varsayılan TTL değerleri kullanır:

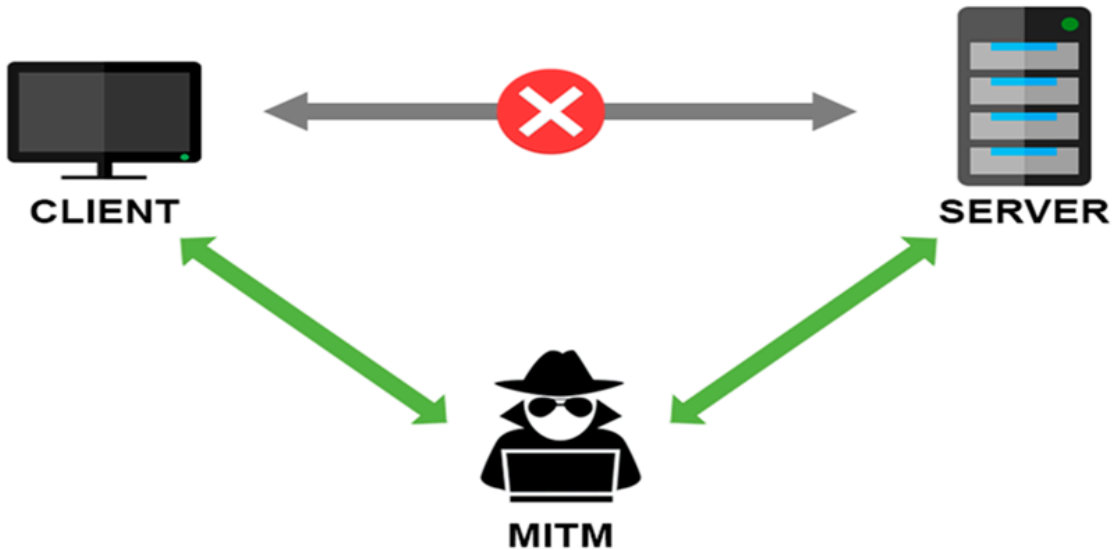
- **Linux / Android / Unix:** 64
- **Windows:** 128
- **Cisco Router / Network Cihazları:** 255

Yaptığımız analizde hedef sistemden dönen paketin TTL değeri **64** olarak tespit edilmiştir.

1. **İşletim Sistemi:** Değerin 64 olması, hedef sistemin **Linux/Unix tabanlı** olduğunu göstermektedir.
2. **Topoloji:** Varsayılan değer (64) ile gelen değer (64) arasında fark olmaması ($64-64=0$), hedefin bizimle **aynı yerel ağda (LAN)** bulunduğunu ve arada hiçbir yönlendirici olmadığını kanıtlar.. Bu tür bilgiler, bir saldırgan için hedef sistem hakkında ön bilgi edinme (işletim sistemi tahmini) amacıyla kullanılabilirken, bir güvenlik analisti için anomali ve spoofing tespiti açısından önemli ipuçları sunar.

4. ARP Spoofing

ARP protokolü, IP adreslerini MAC adresleriyle eşleştirmek için kullanılır. Yerel ağda (LAN) iletişim IP adresleri üzerinden değil, **MAC adresleri** üzerinden yürür. IP ve MAC eşleşmesini ise **ARP (Address Resolution Protocol)** sağlar.



ARP protokolünde doğrulama yoktur. Bir cihaz "Bu IP adresi bana ait" dediğinde, diğer cihazlar buna sorgusuz sualsiz inanır. Bu durum, saldırganın kendisini ağ geçidi (router) gibi tanıtmaya olanak tanır. Kurban sistem bu sahte bilgiyi kabul ettiğinde, tüm trafik saldırgan üzerinden geçer ve **Man-in-the-Middle** saldırısı gerçekleşir.

Günlük hayatta bu durum, apartman girişindeki zil listesinin bozuk olması gibidir: İsteyen kişi "Ben 5 numarayım" dediğinde kapı açılır.

ARP ağda tam olarak bu şekilde çalışır.

Ağ paketlerinin yapısını anlamak, siber güvenliğin temel taşlarından biridir. Kapsülleme süreci, TTL değerleri ve ARP gibi protokoller; hem saldırganlar hem de savunmacılar için kritik bilgiler barındırır.

Wireshark gibi araçlar sayesinde bu küçük başlık alanları incelenerek, ağ üzerindeki anormal davranışlar tespit edilebilir ve olası saldırılar erkenden fark edilebilir. Bu nedenle paket analizi, ağ güvenliği alanında vazgeçilmez bir beceridir.

Kaynakça

- **TryHackMe.** "Network Fundamentals".
<https://tryhackme.com/module/network-fundamentals>
- **TryHackMe.** "Network Traffic Analysis".
<https://tryhackme.com/module/network-traffic-analysis>
- **Cloudflare Learning.** "What is the OSI Model?".
<https://www.cloudflare.com/learning/ddos/glossary/osi-model/>
- **GeeksforGeeks.** "Time to Live (TTL) in Networking".
<https://www.geeksforgeeks.org/time-to-live-ttl-in-computer-network/>
- **Fortinet Cyber Glossary.** "What is ARP Spoofing?".
<https://www.fortinet.com/resources/cyberglossary/arp-spoofing>